MCMP117 · WEEK 7

Fundamentals of Operating Systems

Week 7 · Virtual Memory & File Systems

How can a 4 GB program run in 2 GB of RAM — and where do files really live?

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

LAST WEEK

Week 6 in Four Ideas

Two kinds of address

Programs use logical addresses; the MMU translates them into physical addresses at run time.

Contiguous allocation

One block of RAM per process is simple, but leaves holes: external fragmentation.

Paging

Pages (logical) map to frames (physical) through a page table: no external fragmentation.

TLB

A small cache of page-table entries. Effective access time depends on the hit ratio α.

Last week every page of a process was in RAM. Today we drop that assumption: a page may also be on disk.

Today's Agenda

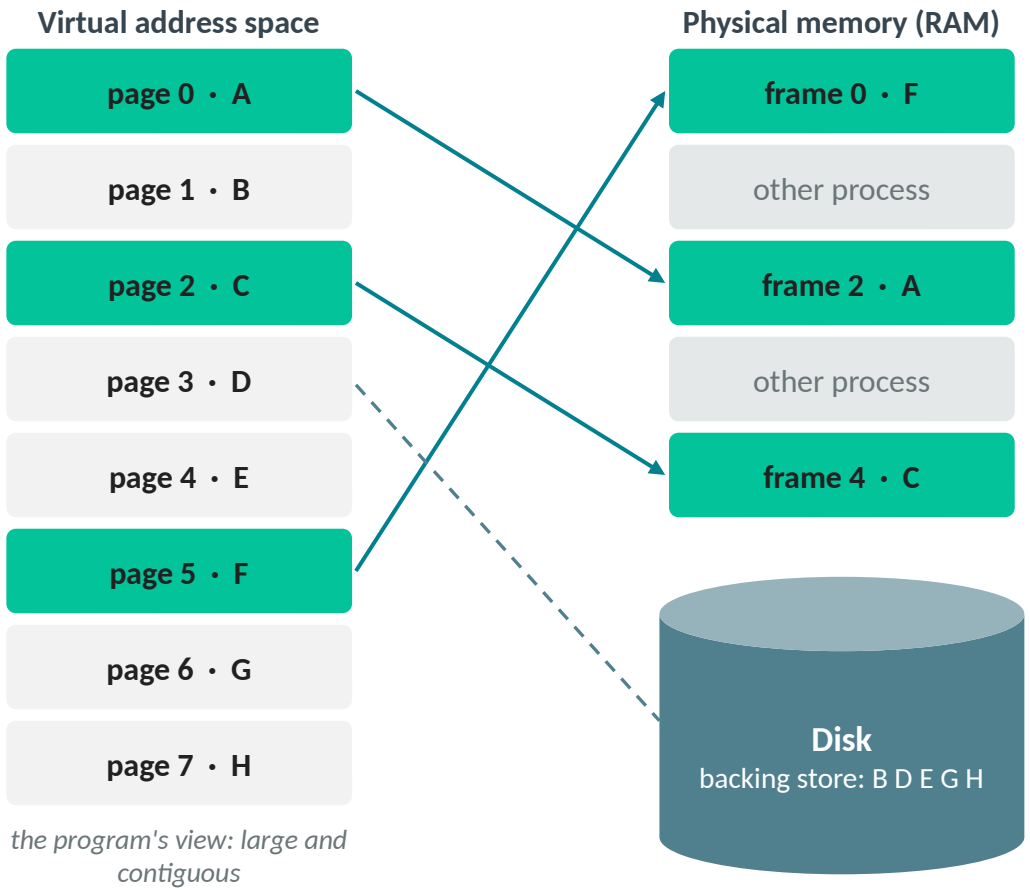
1. **Virtual memory** Only part of a program needs to be in RAM: demand paging and page faults.
2. **Page replacement** When memory is full, who leaves? FIFO, OPT, LRU, Belady's anomaly, the clock.
3. **Thrashing** Too many processes, too few frames — and the working-set idea.
4. **File systems** Files, attributes, operations, access methods.
5. **Directories & Linux** Directory structures, paths, inodes, hard and symbolic links.
6. **Midterm checklist** What to review for next week's exam (Weeks 1–7).

PART 1

Virtual Memory

A program does not need all of its pages in RAM to run.

The Idea: Separate the Program's View from RAM



the program's view: large and contiguous

Why it works

- Much of a program is rarely used: error handlers, unusual menu options, arrays declared larger than needed.
- **Locality:** in any short period, a program uses only a few of its pages.
- **So:** keep only the pages in use in RAM; the rest wait on disk.
- **Benefits:** programs larger than RAM can run; more processes fit in memory (higher CPU utilisation); programs start faster because less is loaded.

Load Everything, or Load Only What Is Needed?

Demand paging = paging + lazy loading: a page is brought into RAM only when it is first used.

Eager: load the whole program

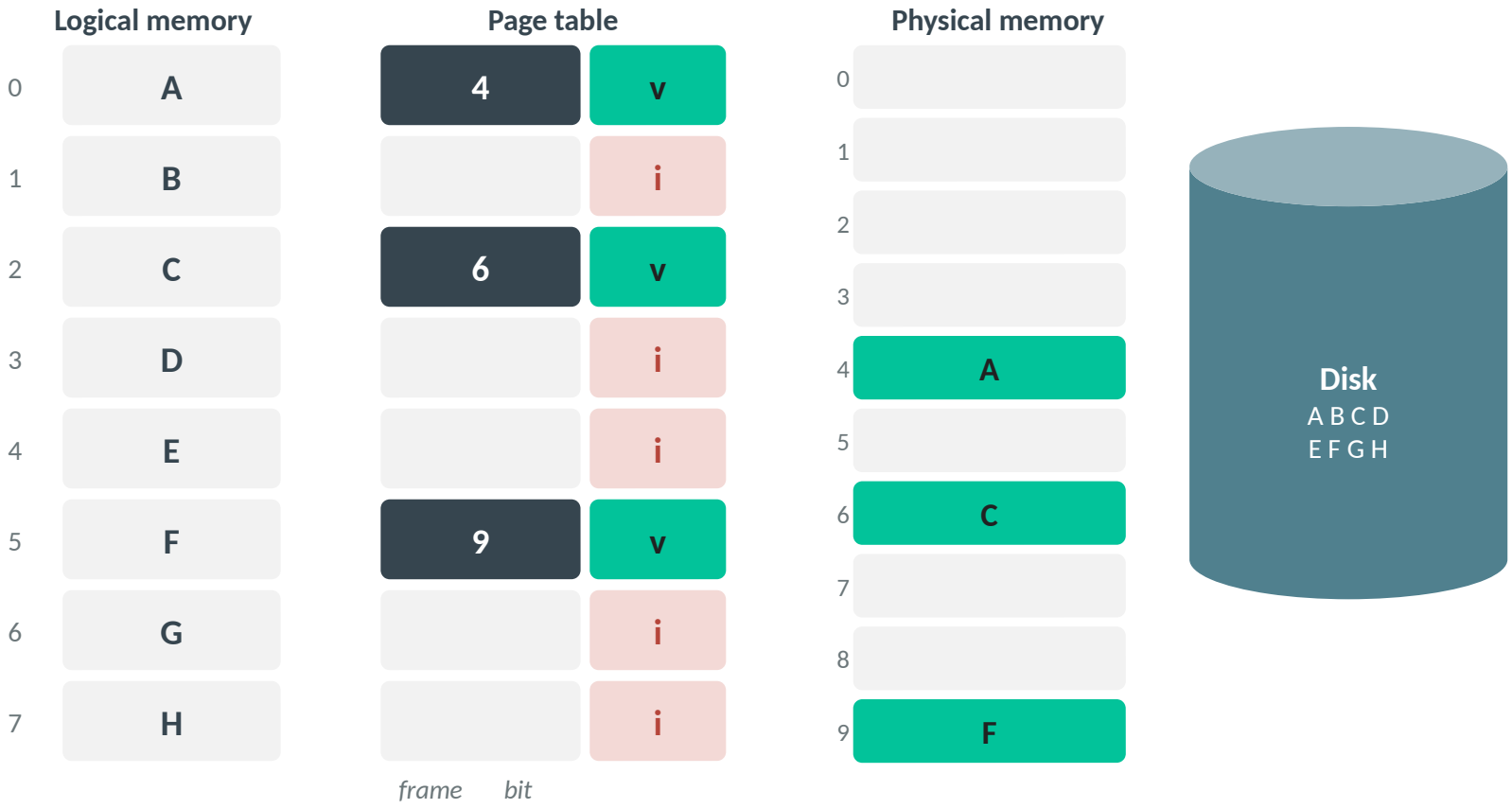
- All pages are read from disk before the program starts.
- Slow start, even if the user only opens one menu.
- Wastes RAM on pages that are never used.
- Fewer programs fit in memory at the same time.

Lazy: demand paging

- Start with few (or zero) pages in RAM.
- Load a page the first time it is referenced.
- Less I/O, less memory, faster start, more processes.
- Price: the first use of each page causes a page fault.

The part of the OS that moves pages between disk and RAM is the pager. "Pure" demand paging starts a program with no pages in memory at all.

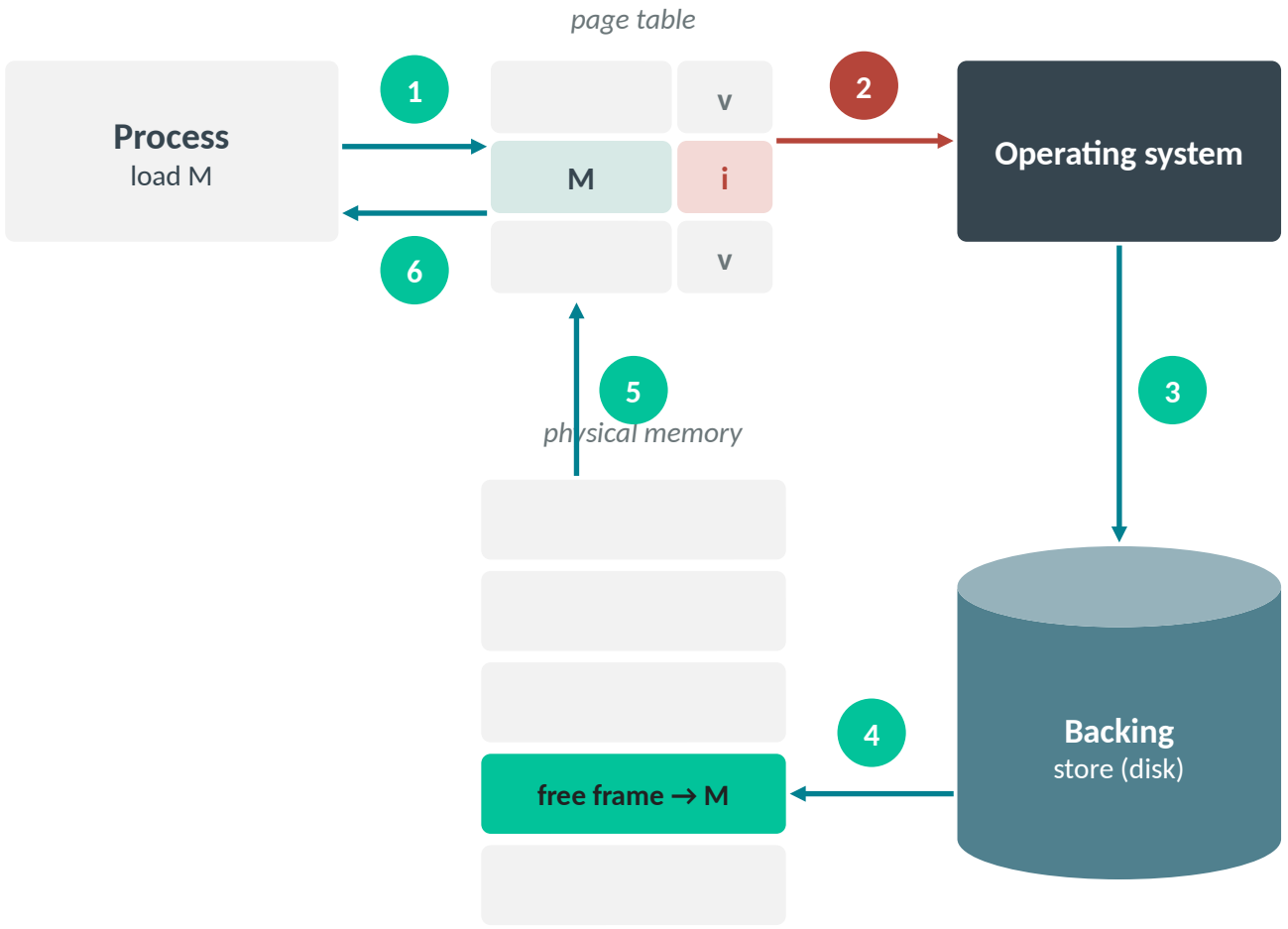
The Valid–Invalid Bit: Is the Page in Memory?



Reading the bit

- **v (valid):** the page is legal and is in a frame.
- **i (invalid):** the page is not in memory (or is not part of the program at all).
- Initially every entry is i.
- Touching an i page makes the MMU trap to the OS: a page fault.

Handling a Page Fault, Step by Step



The six steps

- **1 Reference:** the process touches page M; its bit is i.
- **2 Trap:** the MMU raises a page fault; the OS checks the address is legal (if not: crash).
- **3 Locate:** the OS finds page M on the disk and a free frame.
- **4 Read:** disk → frame (milliseconds; meanwhile another process runs).
- **5 Update:** page-table entry = frame number, bit = v.
- **6 Restart:** the same instruction runs again — now it succeeds.

PERFORMANCE

Effective Access Time with Page Faults

Let p be the page-fault rate ($0 \le p \le 1$): the probability that one memory access causes a page fault.

The formula

- **EAT = (1 − p) × t_m + p × t_fault**
- **t_m:** memory access time (here 200 ns).
- **t_fault:** page-fault service time: trap, disk read, restart (here 8 ms, dominated by the disk).
- A weighted average, like the TLB formula of Week 6.

Lesson: the page-fault rate must be tiny. Locality is what makes it tiny.

```
tₘ = 200 ns,  t_fault = 8 ms = 8,000,000 ns

# Example: p = 0.001 (1 fault per 1000 accesses)
EAT = 0.999 × 200 + 0.001 × 8,000,000
    = 199.8 + 8,000 = 8,199.8 ns ≈ 8.2 µs
→ 41 times slower than RAM!

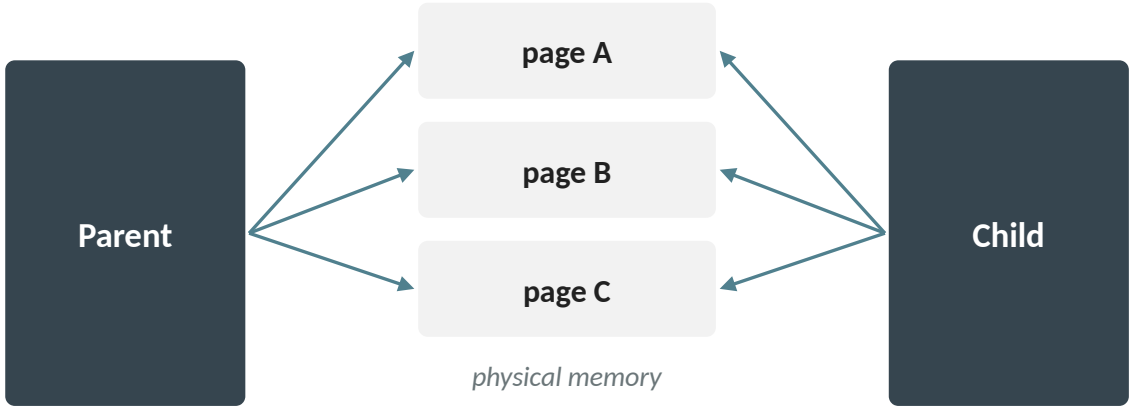
# Slow-down below 10 %  ⇔  EAT < 220 ns
200 + p × (8,000,000 − 200) < 220
p < 20 / 7,999,800 ≈ 2.5 × 10⁻⁶
→ at most 1 fault per ≈ 400,000 accesses
```

VIRTUAL MEMORY TRICK

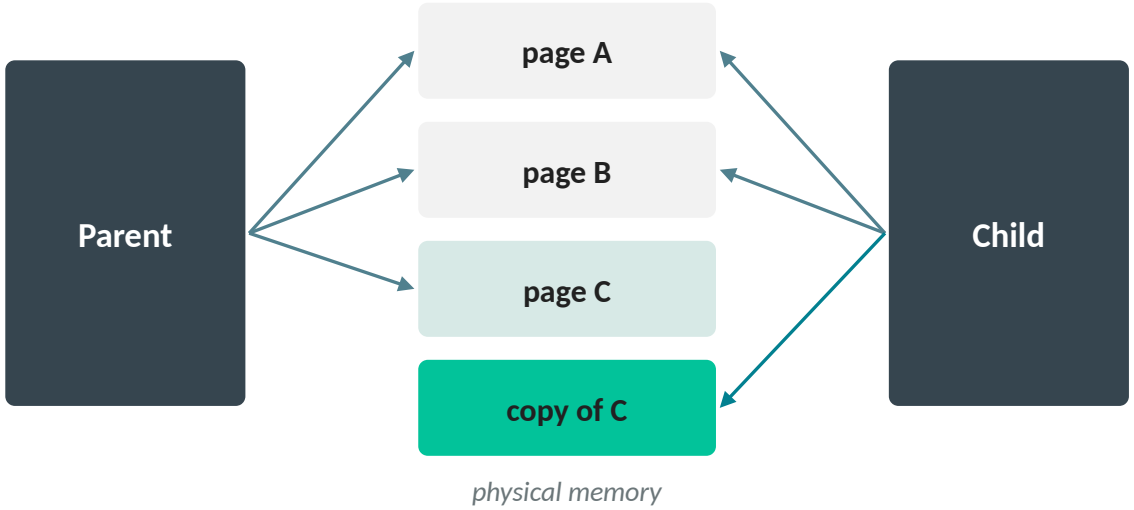
Copy-on-Write: Making fork() Cheap

Week 3: fork() creates a child that is a copy of its parent. Copying every page would be slow — and is often wasted.

1. Right after fork(): pages shared, read-only



2. Child writes page C: only C is copied



After fork() both processes share all pages, marked read-only. The first write to a page causes a page fault; the OS then copies just that page. Pages that are never written — often most of them, especially if the child soon calls exec() — are never copied.

HANDS-ON

Example 1: Watching Demand Paging Happen

```
static long minor_faults(void)          /* kernel's counter */
{
    struct rusage ru;
    getrusage(RUSAGE_SELF, &ru);
    return ru.ru_minflt;
}

int main(void)
{
    long page = sysconf(_SC_PAGESIZE);  /* 4096 bytes  */
    long size = 64 * MB;                /* 16384 pages */

    long f0 = minor_faults();
    char *a = malloc(size);             /* virtual only */
    long f1 = minor_faults();
    for (long i = 0; i < size; i += page)
        a[i] = 1;                       /* first touch  */
    long f2 = minor_faults();
    for (long i = 0; i < size; i += page)
        a[i] += 1;                      /* now resident */
    long f3 = minor_faults();
    /* print f1-f0, f2-f1, f3-f2 */
}
/* full program: code/page_faults.c */
```

What to observe

- malloc() only reserves virtual space: almost no faults.
- The first write to each page faults once: 16,383 + 1 = 16,384 pages.
- Second pass: 0 faults — the pages are now resident.

```
$ ./page_faults
page size 4096 bytes; array 64 MB = 16384 pages
after malloc       :      1 new faults
first touch (write):  16383 new faults
second touch       :      0 new faults
```

Example 2: Copy-on-Write, Measured

```
char *buf = malloc(SIZE);              /* 32 MB = 8192 pages */
memset(buf, 'A', SIZE);                /* parent owns them   */

pid_t pid = fork();
if (pid == 0) {                        /* ---- child ----    */
    long f0 = minor_faults();
    for (long i = 0; i < SIZE; i += page)
        sum += buf[i];                 /* READ: shared       */
    long f1 = minor_faults();
    for (long i = 0; i < SIZE; i += page)
        buf[i] = 'B';                  /* WRITE: copy page   */
    long f2 = minor_faults();
    /* print f1 - f0 and f2 - f1 */
    exit(0);
}
wait(NULL);
printf("parent still sees '%c'\n", buf[0]);
/* full program: code/cow_fork.c */
```

What to observe

- Reading shared pages: no faults at all.
- Writing: one fault per page (8192) — each is copied on first write.
- The parent's data is untouched.

```
$ ./cow_fork
buffer: 8192 pages of 4096 bytes
child read  all pages:      0 faults
child write all pages:   8192 faults
                       (copy-on-write)
parent still sees  'A'
```

No Free Frame? Someone Has to Leave

Memory is over-allocated on purpose. When a page fault finds every frame in use, the OS must choose a victim page.

1. Find the page

Locate the wanted page on the disk.

→

2. Choose a victim

No free frame → a replacement algorithm picks one.

→

3. Write it out?

Only if the victim is dirty (modified).

→

4. Read it in

Load the wanted page; update both page tables.

→

5. Restart

Resume the process that faulted.

The modify (dirty) bit

Hardware sets it when a page is written. A clean victim (e.g. program code) already has an identical copy on disk, so it can be dropped: one disk transfer instead of two.

How we compare algorithms

Run each one on the same reference string (the sequence of page numbers a program touches) with the same number of frames, and count page faults. Fewer is better.

FIFO: Replace the Oldest Page

First-In, First-Out: the victim is the page that has been in memory the longest. Easy to build with a queue.

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 · 3 frames

reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
frame 0	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
frame 1		0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
frame 2			1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
fault?	F	F	F	F		F	F	F	F	F	F			F	F			F	F	F

Note

FIFO ignores how often a page is used: at reference 7 it throws out page 0 — which is needed again immediately (reference 11). Mint = page just loaded; pale columns = faults.

15
page faults with FIFO

OPT: Replace the Page Used Farthest in the Future

Optimal (Belady, 1966): the victim is the page that will not be used for the longest time. Lowest possible number of faults.

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 · 3 frames

reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
frame 0	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
frame 1		0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
frame 2			1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
fault?	F	F	F	F		F		F			F			F				F		

Catch

OPT needs to know the future reference string, so no real OS can implement it. It is the yardstick: "our algorithm is within 10 % of optimal".

9

page faults with OPT (the minimum)

LRU: Replace the Least Recently Used Page

Least Recently Used: the victim is the page that has not been used for the longest time. Uses the past as a guess for the future.

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 · 3 frames

reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
frame 0	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
frame 1		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
frame 2			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
fault?	F	F	F	F		F		F	F	F	F			F		F		F		

Implementation

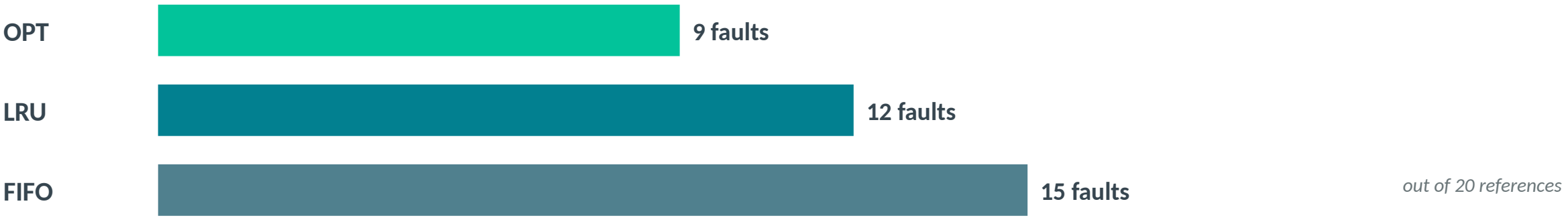
Exact LRU needs a timestamp or a stack updated on every memory access — too expensive in hardware. Real systems approximate it (clock algorithm, coming up).

12

page faults with LRU

Three Algorithms, One Reference String

Algorithm	Victim is the page…	Faults (3 frames)	Implementable?	Belady's anomaly?
FIFO	loaded longest ago	15	yes, very cheap	possible
OPT	used farthest in the future	9	no (needs the future)	never
LRU	used longest ago	12	approximately	never



Ranking on this string: OPT (9) < LRU (12) < FIFO (15). Every algorithm pays the 3 compulsory faults for the first use of 7, 0 and 1.

SURPRISE

Belady's Anomaly: More Frames, More Faults?

FIFO on 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

3 frames → 9 faults

reference	**1**	**2**	**3**	**4**	**1**	**2**	**5**	**1**	**2**	**3**	**4**	**5**
frame 0	**1**	1	1	**4**	4	4	**5**	5	5	5	5	5
frame 1		**2**	2	2	**1**	1	1	1	1	**3**	3	3
frame 2			**3**	3	3	**2**	2	2	2	2	**4**	4
fault?	F	F	F	F	F	F	F			F	F	

4 frames → 10 faults

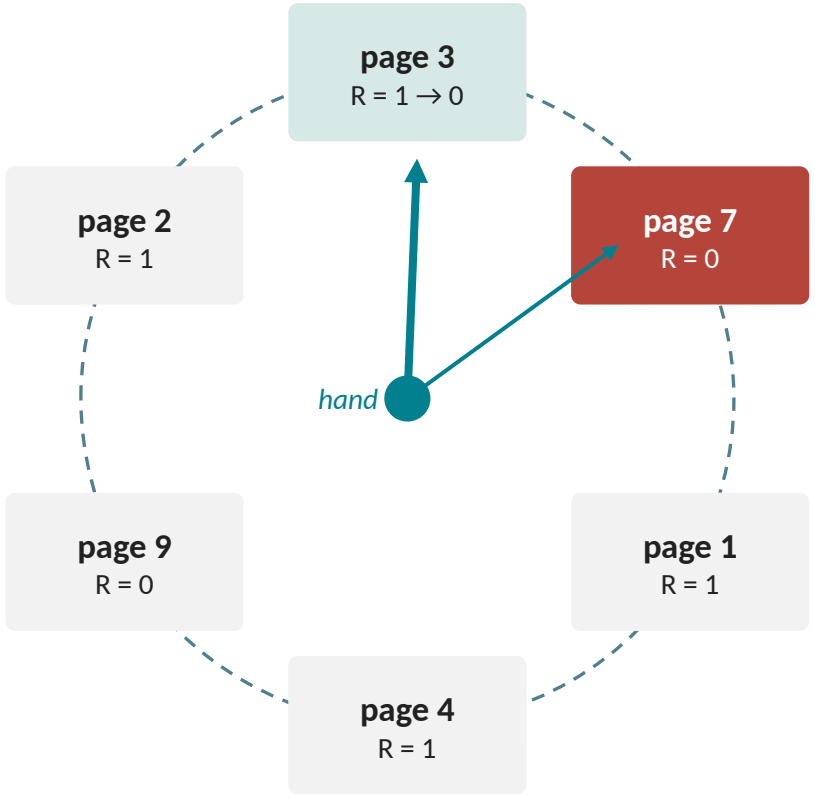
reference	**1**	**2**	**3**	**4**	**1**	**2**	**5**	**1**	**2**	**3**	**4**	**5**
frame 0	**1**	1	1	1	1	1	**5**	5	5	5	**4**	4
frame 1		**2**	2	2	2	2	2	**1**	1	1	1	**5**
frame 2			**3**	3	3	3	3	3	**2**	2	2	2
frame 3				**4**	4	4	4	4	4	**3**	3	3
fault?	F	F	F	F			F	F	F	F	F	F

What it means

- Common sense: more memory should never hurt.
- With FIFO it can: 9 faults with 3 frames, 10 with 4.
- OPT and LRU never behave like this (they are "stack algorithms").
- On the same string LRU gives 10 → 8 faults.

Reference Bit and the Second-Chance (Clock) Algorithm

Hardware sets a page's reference bit R to 1 whenever the page is used. The OS can read and clear it.

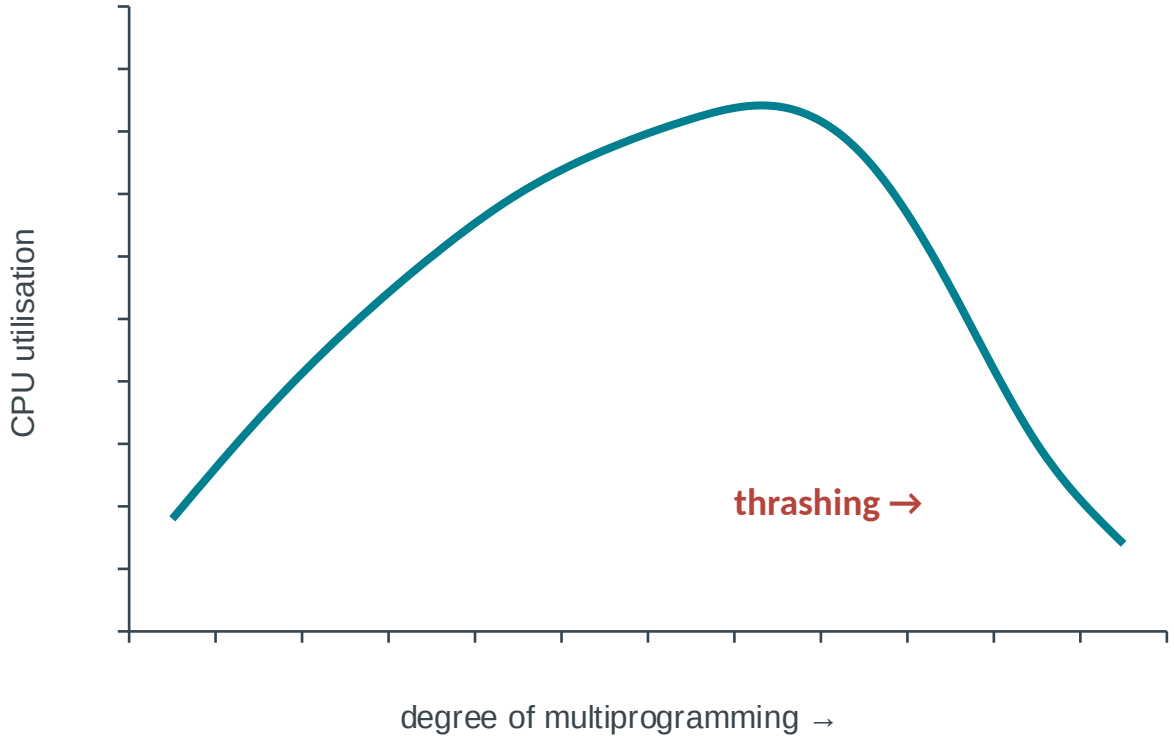


The algorithm

- Frames form a circle; a "clock hand" points to the next candidate.
- **R = 1:** give a second chance — clear R to 0, move on.
- **R = 0:** not used recently → this is the victim.
- **Example:** the hand is at page 3 (R = 1): clear it, move to page 7 (R = 0): replace page 7.
- If every R is 1, the hand goes all the way round: FIFO.

Thrashing: Busy Paging, No Real Work

If a process has fewer frames than the pages it is actively using, it faults again and again.



conceptual curve — not measured data

The vicious circle

- Too many processes → too few frames each → many page faults.
- Processes wait for the disk → CPU utilisation drops.
- A naive scheduler sees an idle CPU and admits more processes…
- …which makes it worse. The system spends its time paging, not computing.
- **Cure:** reduce the degree of multiprogramming (suspend processes) or add RAM.

The Working-Set Model

Working set WS(t, Δ) = the set of distinct pages referenced in the last Δ references. It follows the program's locality.



Rule of the OS

- Give each process at least |WS| frames (its working-set size).
- Total demand D = Σ $|WS_i|$. If D > number of frames m: thrashing ahead → suspend a process.

Choosing Δ

- Δ too small: misses part of the locality.
- Δ too large: covers several localities, wastes frames.
- Simpler cousin: watch the page-fault frequency and give frames to processes that fault too often.

INTERMISSION

10-Minute Break

Next: file systems — files, access methods, directories and Linux links.

PART 2

File Systems

The most visible part of the operating system.

A File and Its Attributes

A file is a named collection of related information stored on secondary storage — the smallest unit a user can save.

Attribute	Meaning	Example (Linux, notes.txt)
Name	Human-readable; in Linux kept in the directory, not with the other attributes	notes.txt
Identifier	Unique number inside the file system	inode 368662
Type	Regular file, directory, link, device, …	regular file
Location	Where the data blocks are on the device	block pointers in the inode
Size	Current size in bytes (or blocks)	6 bytes (8 blocks of 512 B)
Protection	Who may read, write, execute	-rw-rw-r-- owner student
Timestamps	Creation, last access, last modification	Access / Modify / Change

Run stat notes.txt and you see almost every row of this table. The OS keeps these attributes in a file control block (in Linux: the inode).

Eight Basic Operations on a File

create

Find space; add a directory entry. open(..., O_CREAT)

open

Look up the name once; get a file descriptor. open()

read

Copy bytes from the current position. read()

write

Copy bytes to the current position. write()

seek

Move the current position — no I/O. lseek()

close

Release the open-file table entry. close()

delete

Remove the name; free space when unused. unlink()

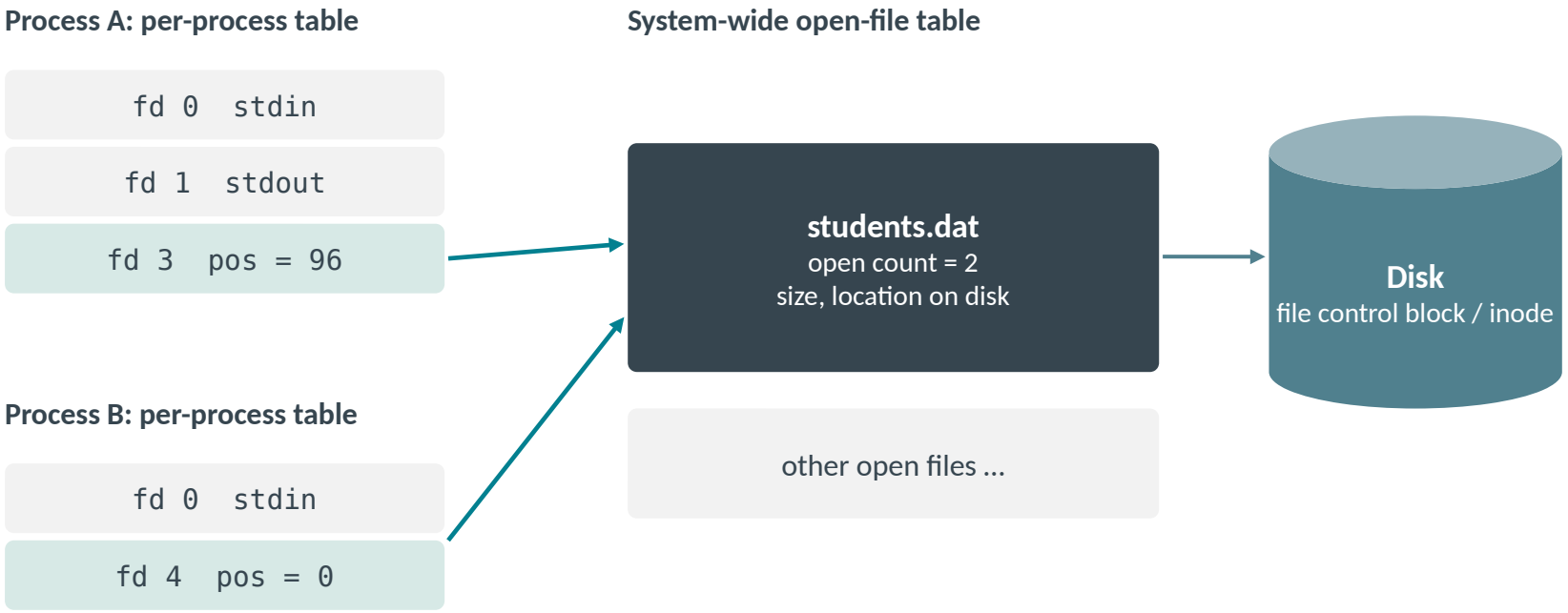
truncate

Erase contents, keep attributes. truncate()

Open once, then use the small integer (file descriptor) for every read/write: the OS does not have to search the directory again. 0, 1, 2 are already open: stdin, stdout, stderr (Week 2).

The Open-File Tables

open() does the slow work once (directory search, permission check) and remembers the result in tables.



Who keeps what

- **Per process:** the file pointer (current position) and access mode.
- **System-wide:** information shared by all users of the file, and an open count.
- close() decrements the count; at 0 the entry is removed.

File Types and Extensions

File type	Usual extensions	Contents
Source code	.c .py .java	program text
Object / executable	.o .exe (none on Linux)	machine code
Script	.sh .bat	commands for a shell
Text / document	.txt .pdf .docx	text, formatted documents
Multimedia	.jpg .mp3 .mp4	image, sound, video
Archive	.zip .tar.gz	several files, compressed

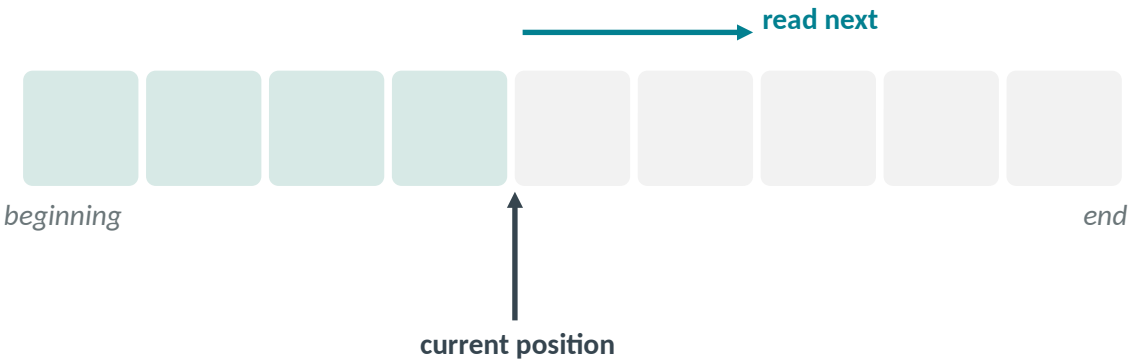
```
$ file file_access.c
file_access.c: C source, ASCII text
$ file file_access
file_access: ELF 64-bit LSB pie executable,…
$ file photo.txt
photo.txt: JPEG image data, JFIF standard 1…
$ file students.dat
students.dat: data
```

Name vs. content

- Windows uses the extension to choose a program.
- In Linux an extension is only a hint; `file` reads the content ("magic number"). photo.txt is really a JPEG.

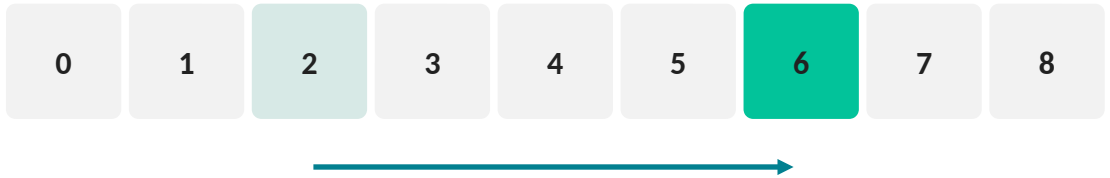
Sequential Access vs. Direct Access

Sequential: read next / write next / rewind



- Information is processed in order, one record after another.
- The model of a tape — and of most programs: compilers, editors, media players.
- In C: repeated read() calls; the position advances by itself.

Direct (relative): read n / write n / seek n



jump from block 2 to block 6 — no reading in between

- The file is a numbered sequence of fixed-size blocks or records.
- Any record can be read in any order — the model of a disk.
- Databases, airline reservations. In C: lseek() then read().

Worked Example: Where Is Record 1000?

The question

- A file holds student records of 100 bytes each, numbered from 0.
- The file system uses blocks of 4096 bytes.
- Which block holds record 1000, and where in that block does it start?

```
byte offset = record number × record size
            = 1000 × 100 = 100,000

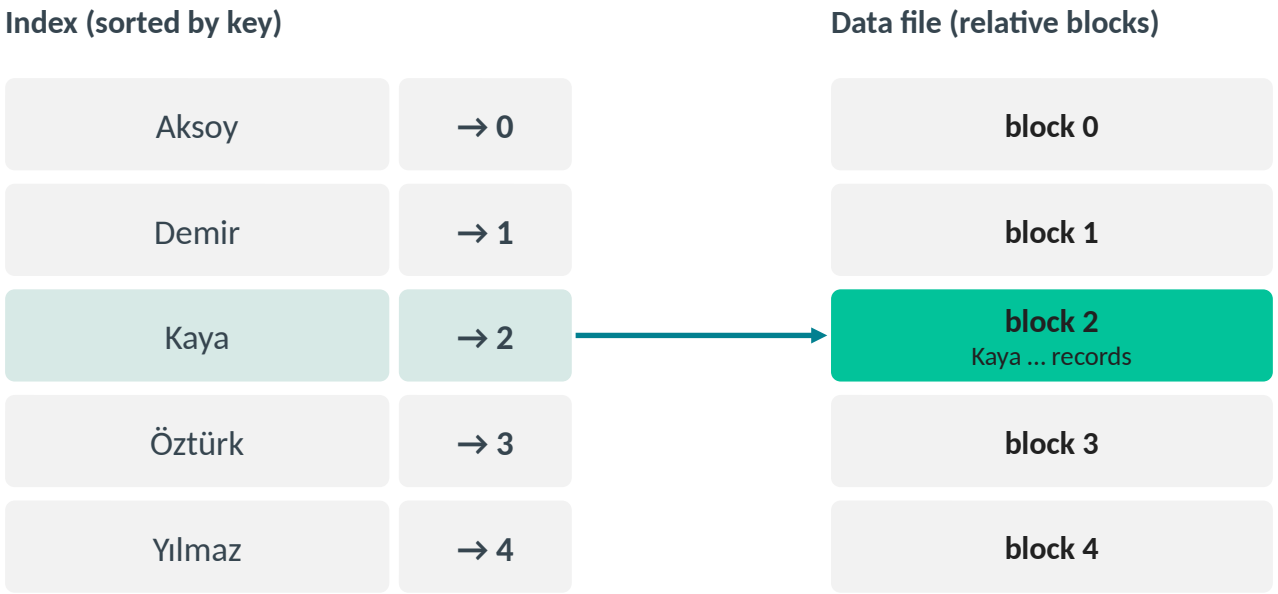
block       = ⌊100,000 / 4096⌋ = 24
offset      = 100,000 − 24 × 4096
            = 100,000 − 98,304 = 1,696

last byte   = 100,099  → still block 24
```

So: read relative block 24 and take bytes 1,696 … 1,795. In C this is one line: lseek(fd, 1000 * 100, SEEK_SET); then read(fd, rec, 100). The OS turns the byte offset into a block number exactly as above. Our demo program uses 32-byte records: record 3 starts at byte 3 × 32 = 96.

Indexed Access: Look Up First, Then Jump

A small index file maps keys to block numbers; search the index, then read one block directly.

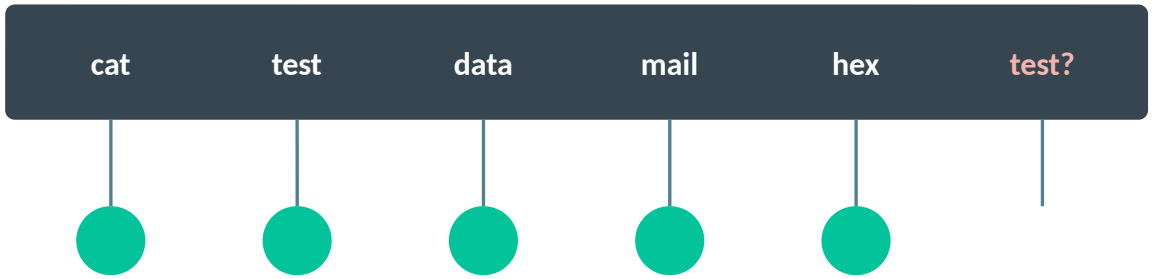


Why and how

- Search a small, sorted index (fast) instead of the whole file.
- Then one direct read of the right block.
- Very large files: an index of the index (multi-level).
- The basis of databases (B-trees) and of how file systems find a file's own blocks.

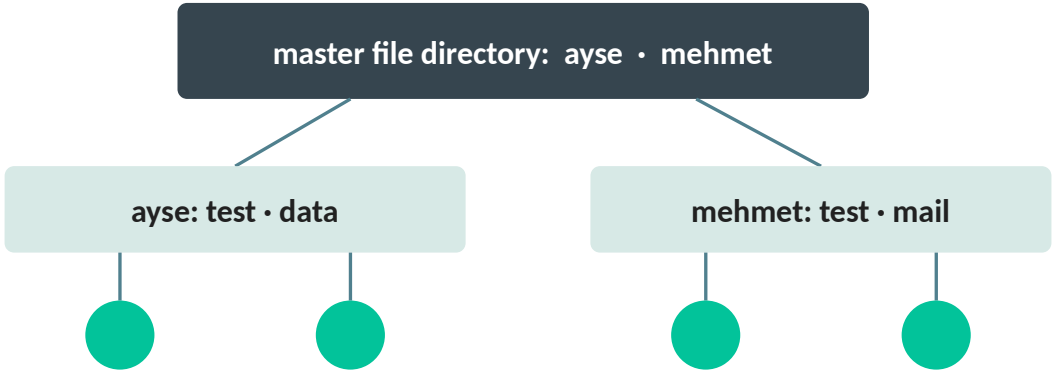
Single-Level and Two-Level Directories

Single-level: one directory for everyone



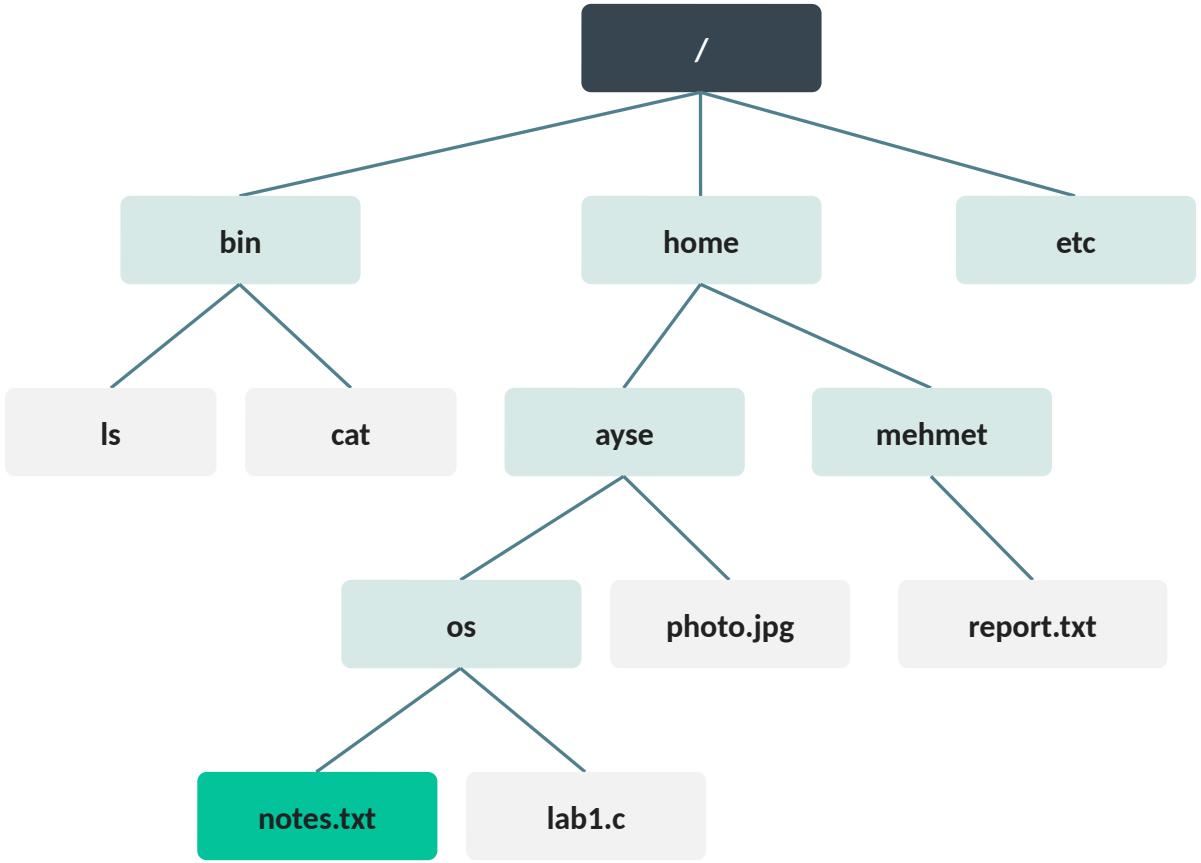
- Simple, but every file name must be unique — across all users.
- Two users cannot both call a file test.
- Hundreds of files in one list: impossible to organise.

Two-level: one directory per user



- Each user has a private directory: both may own a test file.
- Path name = user + file name, e.g. /ayse/test.
- But no subfolders, and sharing files between users is awkward.

Tree-Structured Directories and Path Names



pale = directory, white = file

Two ways to name a file

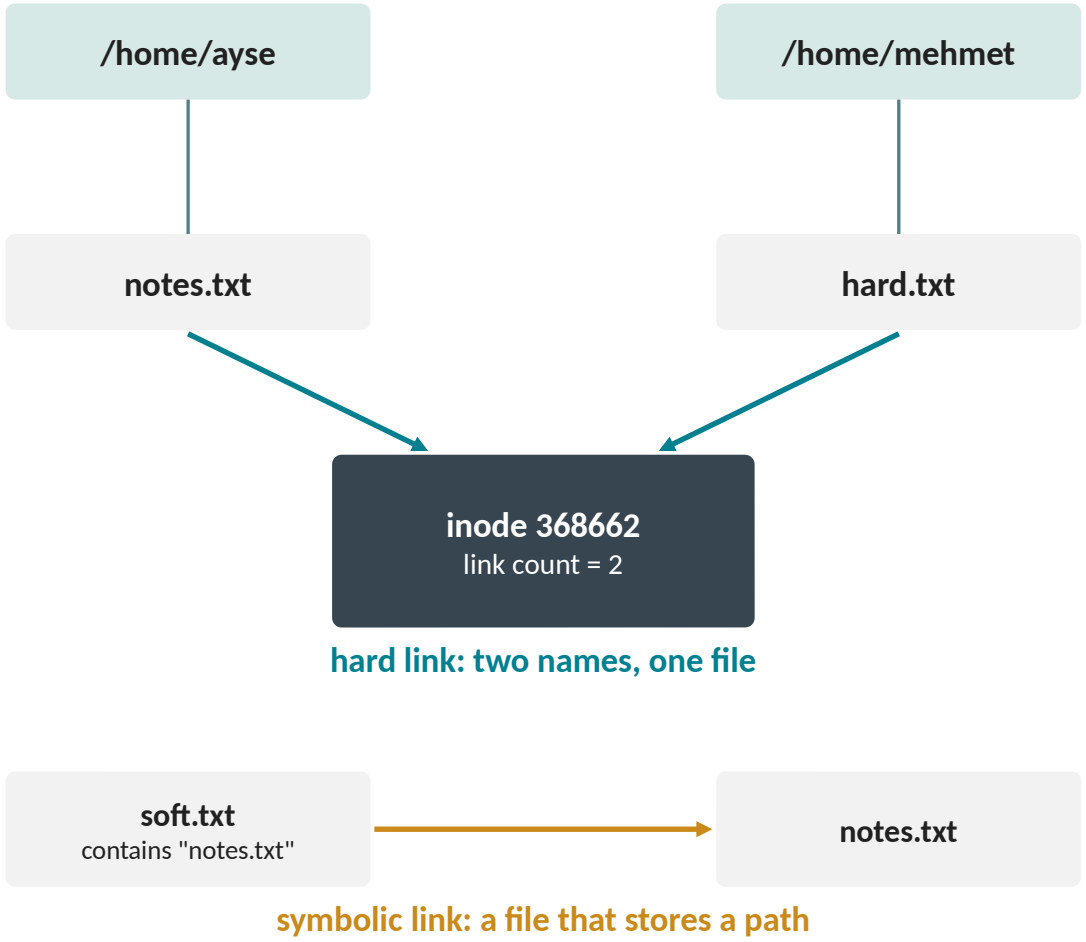
- **Absolute path:** starts at the root /.
- **Relative path:** starts at the current directory.
- **. and ..** mean this directory and its parent.
- Week 9: pwd, cd and ls move you around this tree.

```
# absolute: works from anywhere
/home/ayse/os/notes.txt

# relative, current directory /home/ayse
os/notes.txt

# relative, current directory /home/ayse/os
../../mehmet/report.txt
```

Acyclic Graphs: Sharing with Links



Acyclic graph

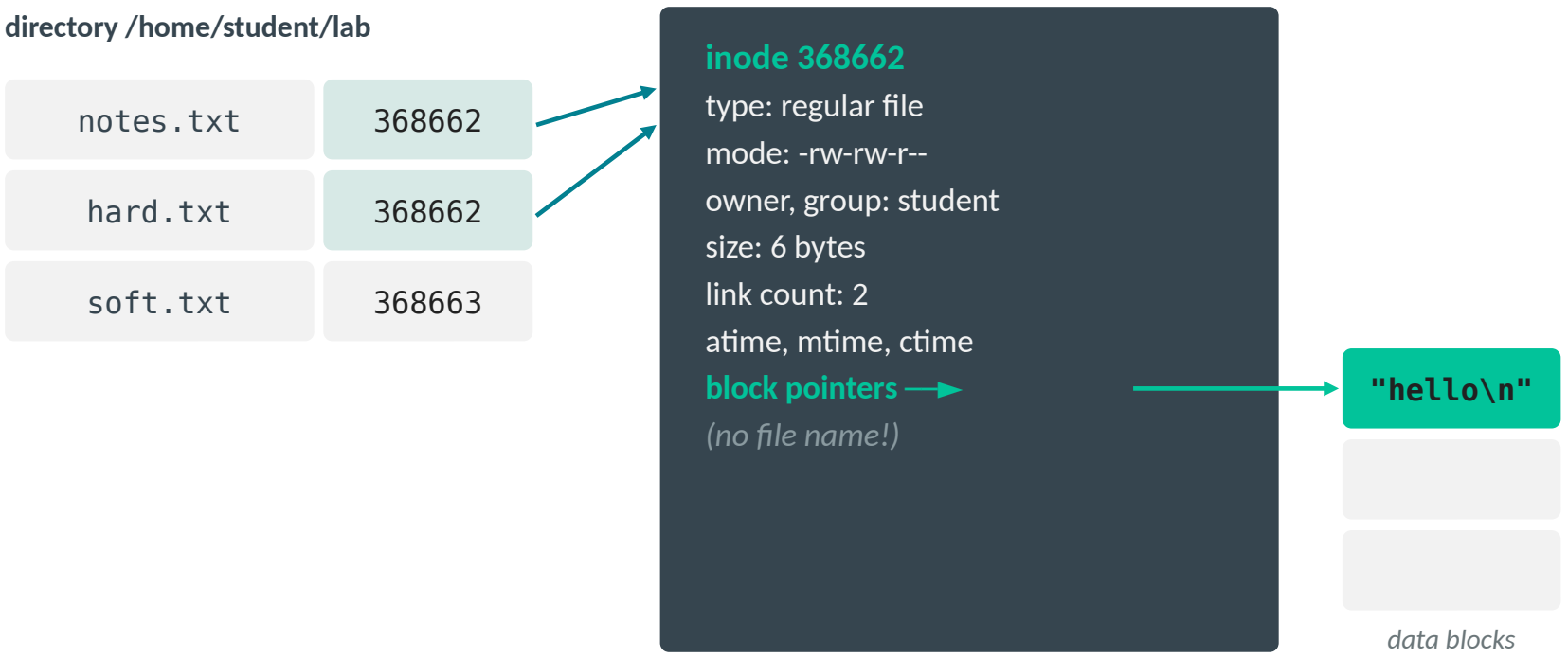
- A file (or directory) may appear in several directories: shared, not copied.
- **Hard link:** another directory entry for the same inode. Deleting one name leaves the file.
- **Symbolic link:** a small file holding a path; it dangles if the target is removed.

General graph (allowing cycles)

- A link back to an ancestor creates a cycle: searches may loop forever, and a deleted subtree may never become "unused".
- Linux forbids hard links to directories; symlink loops stop with an error (ELOOP).

What Linux Shows: Directory Entries and Inodes

A directory is just a table of (name, inode number) pairs. Everything else about a file lives in its inode.



See it yourself

- ls -i: inode numbers.
- ls -l: 2nd column = link count.
- stat file: the whole inode.
- df -i: free inodes.

HANDS-ON

Example 3: ls -li, ln, ln -s and stat

```
$ echo "hello" > notes.txt
$ ln notes.txt hard.txt
$ ln -s notes.txt soft.txt
$ ls -li
368662 -rw-rw-r-- 2 student student 6 Oct  8 12:20 hard.txt
368662 -rw-rw-r-- 2 student student 6 Oct  8 12:20 notes.txt
368663 lrwxrwxrwx 1 student student 9 Oct  8 12:20 soft.txt -> notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 368662      Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1001/ student)   Gid: ( 1002/ student)
Modify: 2026-10-08 12:20:48.530146698 +0000
Change: 2026-10-08 12:20:48.534146698 +0000
$ rm notes.txt
$ cat hard.txt
hello
$ cat soft.txt
cat: soft.txt: No such file or directory
$ ls -li
368662 -rw-rw-r-- 1 student student 6 Oct  8 12:20 hard.txt
368663 lrwxrwxrwx 1 student student 9 Oct  8 12:20 soft.txt -> notes.txt
```

What to observe

- notes.txt and hard.txt: same inode, link count 2.
- soft.txt: its own inode; size 9 = the length of "notes.txt".
- Change time (ctime) moved: ln updated the inode.
- After rm: hard.txt still works, soft.txt dangles.

HANDS-ON

Example 4: Sequential and Direct Access in C

```c
#define RECSZ 32                        /* bytes per record */
int fd = open("students.dat", O_CREAT | O_RDWR | O_TRUNC, 0644);
/* ... write 5 records: "0 Ayse", "1 Mehmet", ... "4 Zeynep" */

lseek(fd, 0, SEEK_SET);                 /* 1) sequential   */
while (read(fd, rec, RECSZ) == RECSZ)
    printf("  [%s]", rec);

lseek(fd, 3 * RECSZ, SEEK_SET);         /* 2) direct: byte 96 */
read(fd, rec, RECSZ);

lseek(fd, 1 * RECSZ, SEEK_SET);         /* 3) overwrite rec 1 */
write(fd, rec, RECSZ);
close(fd);
/* full program: code/file_access.c */
```

What happens

- read() advances the file pointer by 32 bytes each time: sequential.
- lseek() sets the pointer anywhere; the next read() or write() starts there: direct.
- Writing record 1 does not touch the others.

```
$ ./file_access
sequential:  [0 Ayse]  [1 Mehmet]  [2 Elif]  [3 Can]  [4 Zeynep]
direct: record 3 at byte 96 -> [3 Can]
after update: record 1 -> [1 Mehmet (updated)]
file size: 160 bytes
```

QUICK QUIZ

Check Your Understanding

1. Reference string 1, 2, 3, 1, 4, 1, 2 with 3 frames: how many page faults with FIFO? With LRU?

2. t_m = 200 ns, t_fault = 8 ms, p = 0.0001. What is the effective access time?

3. Records are 64 bytes, blocks 4096 bytes. In which block, at which offset, does record 200 start?

4. After ln a.txt b.txt and rm a.txt, can you still read the data? What if b.txt had been made with ln -s?

Week 7 — Key Takeaways

- **Virtual memory:** a process sees a large address space; only the pages in use are in RAM. Demand paging loads a page on first use.
- **Page fault:** access to an invalid page traps to the OS, which loads the page and restarts the instruction.
  EAT = $(1 - p) \cdot t_m + p \cdot t_fault$, so p must be tiny.
- **Copy-on-write:** fork() shares pages until one side writes.
- **Replacement:** FIFO (simple, Belady's anomaly), OPT (optimal, needs the future), LRU (good, approximated by the clock). Example: 15 / 9 / 12 faults.
- **Thrashing:** too few frames → constant paging; keep each working set in memory.
- **Files:** named data with attributes (in the inode); open-file tables; sequential, direct, indexed access.
- **Directories:** single-level → two-level → tree → acyclic graph (hard and symbolic links).

Midterm Checklist: Weeks 1–7

W1 History

Batch, multiprogramming, time-sharing; why idle CPUs drove OS design; UNIX and Linux.

W2 OS basics

Definitions, goals, interrupts, storage hierarchy, dual mode, system calls, OS structures.

W3 Processes

OS services, system-call types, process states, PCB, context switch, fork().

W4 Threads, CPU

User vs. kernel threads, multithreading models, scheduling criteria.

W5 Algorithms & sync

FCFS, SJF/SRTF, Priority, RR with Gantt charts; race conditions, mutex, semaphores.

W6 Memory

Logical vs. physical, MMU, fragmentation, paging, TLB and EAT, segmentation.

W7 VM & files

Page faults, EAT with p, FIFO/OPT/LRU tables, thrashing; access methods, directories, links.

How to prepare

Redo every worked example and quiz from the slides and the notes' review questions.

NEXT WEEK

Midterm Exam (Week 8)

Week 8 will cover:

- Covers Weeks 1–7: slides plus the course notes
- Practise: Gantt charts, EAT (TLB and page faults), FIFO/OPT/LRU tables, block arithmetic
- Bring your student ID, pens and a basic calculator if allowed
- Time and room: check the course Telegram channel
- After the midterm, Week 9 starts hands-on Linux: the shell and navigation

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz